# 5th April : Chai aur SQL Part-2



## #Joins in SQL

Joins are used to combine rows from two or more tables based on a related column.

## # Types of Joins in SQL

1. LEFT JOIN (LEFT OUTER JOIN)
2. INNER JOIN
3. ~~OUTER JOIN~~ CROSS JOIN
4. FULL JOIN (FULL OUTER JOIN)
5. RIGHT JOIN (RIGHT OUTER JOIN)
6. SELF JOIN

## # Left Join : select all rows from left table, but only matching rows from right table.

01-left-join.sql

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name       VARCHAR (100),
    email      VARCHAR (100)
    branch     VARCHAR (50)
);

CREATE TABLE internships (
    internship_id SERIAL PRIMARY KEY,
    company_name  VARCHAR (100),
    role          VARCHAR (50),
    stipend       INT CHECK (stipend > 1000)
    status        VARCHAR (20)

    student_id INT REFERENCES students (student_id)
    ON DELETE CASCADE NULL
)
```

Here REFERENCES student (student_id), this creates a foreign key.
↳ Meaning student_id must exists in both tables

2

# ON DELETE CASCADE

Here if a student is deleted → all their internships also gets deleted automatically

ON DELETE is done to ensure that no internship data is left without student, also keeps db clean.

# ON DELETE SET NULL

If the student is deleted, the internships stay, but the foreign key value is set to NULL

# ON DELETE RESTRICT

If a student gets a internship, it cannot be deleted. Good, as it don't allow deleting a student if their internship exists.

# FOREIGN KEY (FK)

It is a column in one table that points to the primary key of another table.

# LEFT JOIN simply means left table ka saara data aayega, ~~left~~ right table se sirf matching data will come.

## INNER JOIN

Only the tables which are common will be displayed.
It will help me find "only students who got the internship
eg:

```
SELECT
    students.name,
    students.branch,
    internships.company_name,
    internships.role,
    internships.~~sti~~ stipend,
FROM students
INNER JOIN internships ON students.student_id = internships.
                                                student_id;
```

# To select all columns,

```
SELECT
    s.student_id as student_table_id,
    i. . . . .
FROM students AS s
INNER JOIN internships AS i ON s.student_id = i.student_id
```

In the part of the student and the internship table.

1. INNER JOIN → Only students who have internships
2. LEFT JOIN ⇒ All students, even if any student has internship or not.
3. RIGHT JOIN ⇒ opposite of left join
   All internships will come, even if any student is missing
4. FULL JOIN / ⇒ Everything from both tables will come

   FULL OUTER JOIN

Don't prefer right join, just swap tables and use it

eg.



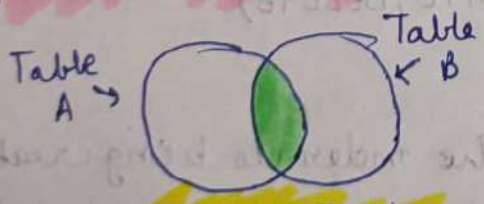
```
SELECT
    s.name
    s.branch
    i.company_name
    i.stipend
FROM internships AS i LEFT JOIN students AS s ON
i.student_id = s.student_id;
```

This condⁿ is checked while joining.

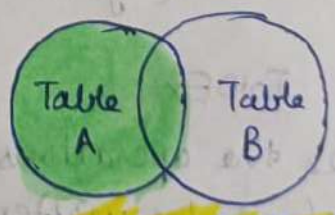
# FULL OUTER JOIN

```
SELECT
    s.name AS student_name,
    s.branch
    i.company_name,
    i.status
FROM students s
FULL OUTER JOIN internships i ON s.student_id = i.student_id
```

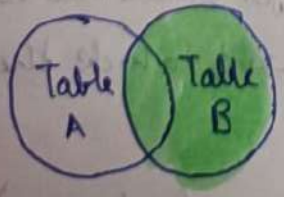
# VENN DIAGRAM




INNER JOIN




LEFT JOIN




RIGHT JOIN

FULL JOIN

Q What is DB Index ?

DB index actually tells DB ki jo data aapko chahiye woh memory ke kis kone mein hai.

Without index, db has to scan every row, but using an index DB directly jumps to a particular index

# EXPLAIN ANALYZE

It shows how PostgreSQL executes our query.
It shows which method was used (Index VS scanning)
- how many rows are checked
- total execution time

# PARALLEL SEQUENTIAL SCAN

A db scans entire table, but splits work across multiple workers to make this faster.

# CREATE INDEX CONCURRENTLY (in PostgresSQL)

This means, we can create an INDEX without blocking normal operations (eg, INSERT, UPDATE, DELETE)

# CREATE INDEX

This blocks the operations while the index is being created DB is locked for writes, so other users cannot modify data in the meanwhile. Good for small tables, as in ~~eg~~, real life scenario, we don't want to lock the DB.

eg

```
CREATE INDEX idx-name ON marks (name);
```

Q What is an Index Scan?

When a DB uses an index to directly find rows, instead of scanning whole table.

6

⇒ Index is usually a sorted B-tree.

⇒ DB performs fast lookup

⇒ Gets the rows location using a pointer

Advantages:

⇒ Works best if query is highly selective

⇒ Best if only few rows match

# Indexing

1. Create index;

```
CREATE INDEX idx_name ON marks(name);
```

2. Concurrent index;

```
CREATE INDEX CONCURRENTLY idx_name ON marks(name);
```

3. Remove index;

```
DROP INDEX idx_name;
```

# Non-Key Value Index:

A index created on column that is NOT a primary key. You have to create this manually on other columns.

⇒ This is done to speed up queries on non-primary columns.

Disadvantage:

⇒ Slows down INSERT, UPDATE, DELETE operations

⇒ Extra storage required

⇒ May not be always used, bcs if DB has a better way it will follow that.

# DB decides how to execute our query where it wants to do index scan or sequential scan or parallel scan.



But DB does not automatically or blindly create indexes for us, except primary key, unique constraint etc.

# Transaction

In transaction, there should not be partial success or failure.
just like payment must be success or fail (not partial)

Three states of transaction,

1. BEGIN → start transaction.

   . . . . .

   queries here

2. COMMIT; → saves changes
3. ROLLBACK; → undo all changes

Note: ROLLBACK can't be done after commit, as soon as you do COMMIT, you come out of the transaction, meaning transaction is finished

Dirty Read: If you start a transaction and update some data, BEFORE you COMMIT that transaction, other transactions MAY SEE the changed data. (This is actually not a good thing)

Here we are reading uncommited data.

This happens when one transaction reads data that has been modified by another transaction but not yet committed.

TXN 1

BEGIN;

UPDATE accounts SET amount -500 WHERE name = "Jcani";

UPDATE accounts SET amount +500 WHERE name = "Brother";

COMMIT; → This will commit changes

ROLLBACK; → This will revert the changes done after BEGIN keyword

# ACID Properties:

A (Atomicity) ⇒ All or nothing, even if one query fails all will fail.

For DB, any failure is a rollback, unless you explicitly tell to commit.

C (Consistency) ⇒ (balance will not go to negative) DB always move from one valid state to another (nothing in between) All rules must be followed

I (Isolation) ⇒ Even if many users run queries at the same time, each transaction behaves like it's running alone. Only when a query is finished and its commited other queries get to know about it.

Ready Commited Isolation:

~~Dirty read~~ ⇒ data will be displayed only after commiting, can't read before transaction completes.

D (Durability) ⇒ Once we have COMMITED, data is permanently stored; even if the system crashes, data will be there



# POST ANOMALIES

⇒ Unexpected problems while inserting, updating or deleting data

Types of Anomalies:

1. Insertion Anomaly
2. Update Anomaly
3. Deletion Anomaly

} study more about it.

Any DB which is doing all ACID properties is a ACID compliant database. eg POSTgreSQL, MySQL, Oracle etc

# CHAR VS VARCHAR

CHAR stores fixed number of characters

VARCHAR stores only actual data, so the length is not fixed

# DB Design for instagram:

users [icon : user] {

id SERIAL PK
username varchar (50) unique NOT NULL
email varchar (100) unique NOT NULL
phone char (15)
dob ~~date (15~~ date
password varchar (256)
bio text

avatar_url text
forgot_password_token text
email_verification_token text
created_at timestamp
updated_at timestamp
}

users to posts is one to many

posts [icon: notebook] {
id SERIAL pk
caption text
image_url text NULL
video_url text null
reel_url text null
author_id int (fk) ← foreign key
created_at timestamp
updated_at timestamp
}

stories [icon: notebook] {
id SERIAL pk
image_url text null
video_url text null
post_id int fk null
author_id int fk
created_at timestamp
updated_at timestamp
}

⇒ One user can have many stories

users to story is one to many

~~Onto~~ One story can have one post that is one to one relationship

bookmarks [icon: bookmark] {
  id serial pk
  post_id int fk
  author-id int fk
  created-at timestamp
  updated-at timestamp
}

eg stories.post_id - posts.id // one to one
bookmarks.post_id - posts.id ⇒ One story can have only one post.
// One post can be bookmarked only once

So, users.id < posts.author_id // one to Many
users.id < stories.author_id
users.id < bookmarks.author_id

Means, One user can create a many posts, many stories and many bookmarks

eg stories.post_id > posts.id // many to one
bookmarks.post_id > posts.id // many to one
This means many stories → one post
many bookmarks → one post

comments [icon: message-circle] {
  id serial pk
  author_id int fk
  post_id int fk
  content text
  created-at timestamp
  updated at timestamp
}

users.id – account_setting.id // one to one
One user can have only one account setting

comments.author_id > users.id
comments.post_id > posts.id
~~which means many stories can be posted in one post~~
which means many comments can belong to one user
many comments can belong to one post.

#All these are business requirements, which is bcs nothing is inherently wrong.

likes [icon : heart] {
id serial pk
author_id int fk
post_id int fk
created_at timestamp
updated at timestamp
}

⇒ likes.author_id > users.id
likes.post_id > posts.id

Note : USE FOR UPDATE
This locks rows when we select a row, till the ongoing transaction end
eg.
A seat is selected, will be kept on hold till the select is either selected or payment is done
↳ doing a row level lock

// many to many
Many users like many posts
One user likes many posts
One post can be liked by many users.

// Complete :
follower [icon : users] {
// user 1 can be followed by user 2
3 // user 2 can be followed by user 1